False Sharing and Scalability in Multicore Processors: Impact of Cache-Line Data Layout on Parallel Performance

Walter Alfaro, Allan Arrieta

***Abstract*—Multicore processors keep caches coherent per cache line, not per variable, so threads that write independent variables stored in the same line force that line to move between cores. This effect, false sharing, is well documented, but most guidance about it comes from processor vendors, tool developers and studies of server-class machines. This paper examines how the placement of variables in cache lines affects a parallel program on a hybrid Intel Core i7-1260P and tests common claims from those sources rather than assuming them. We combine a MESI coherence simulator, written for this study and calibrated with our own measurements, with a microbenchmark that compares packed, padded, per-thread and truly shared counters on performance cores and on efficiency cores with and without a shared L2, using `perf c2c` to locate contended lines. With four threads, packed counters were 8.4 to 11.6 times slower than padded ones; the cost grew linearly with the number of threads, at 42 to 50 cycles per line transfer, which the simulator reproduced within 7%. Three claims proved context-dependent: padding to 128 bytes gave no benefit over 64 bytes, a `volatile` demonstration almost hid the problem on performance cores but not on efficiency cores, and `perf c2c` reported 60 to 350 times fewer HITM events on efficiency cores for a comparable slowdown. Per-thread accumulation outperformed padding without extra memory. We conclude with practical guidelines for data layout and for detecting false sharing on hybrid processors.**

***Index Terms*—False sharing, cache coherence, MESI, multicore processors, hybrid processors, cache line, padding, simulation, performance analysis, perf c2c.**

I. Introduction

PARALLEL programs are expected to run faster as more cores are added. A well-known obstacle to this expectation is *false sharing*: threads that never touch each other's data can still slow each other down when their data happen to share a cache line [1], [2]. Because the effect depends on memory layout rather than on the algorithm, it is invisible in the source code and is usually discovered only through profiling [3], [4].

The problem is not only of historical interest. The Linux kernel documentation describes false sharing as a recurring cause of performance regressions in the kernel and lists the data-layout patterns that produce it [2]. At the same time, the hardware on which programs run has changed: since its 12th generation, Intel's client processors combine two kinds of cores, performance cores (P-cores) and efficiency cores (E-cores), with different microarchitectures and cache organizations on the same chip [5]. A thread may run on either kind, so the cost of sharing a line, and the ability of tools to detect it, may differ between two threads of the same program. Most of the classic literature on false sharing predates this design.

Most of the guidance on false sharing comes from a small group of sources: processor vendors, the developers of the detection tools, and research prototypes evaluated by their own authors. Rather than taking that guidance at face value, this paper treats each recommendation as a claim to be tested. We pose three questions:

1) **RQ1:** How much does placing independent counters in the same cache line degrade performance and scalability on a contemporary hybrid laptop processor?
2) **RQ2:** Which commonly cited claims about false sharing (magnitude of the slowdown, need for 128-byte padding, reliability of HITM-based detection) hold under our model and measurements, and which depend on the context in which their sources were produced?
3) **RQ3:** What does mitigation cost in memory footprint, and when is that cost worth paying?

To answer them, this paper makes four contributions:

- a trace-driven MESI simulator, written for this study, that isolates the coherence events caused by each data layout and lets us vary parameters that are fixed in hardware, such as the line size and the adjacent-line prefetcher;
- measurements on an Intel Core i7-1260P comparing four data layouts on three groups of cores (P-cores, E-cores that share an L2, and E-cores in different clusters), complemented with `perf c2c` recordings;
- a comparison between the calibrated simulator and the hardware, in which the disagreements are treated as findings rather than errors;
- a critical reading of the main sources, in which each recommendation is checked against our own evidence and the context in which it was written.

Section II reviews cache coherence and the detection of false sharing. Section III summarizes the sources and their context. Section IV describes the simulator and the hardware experiment, Section V presents the results, Section VI discusses them against the sources, and Section VII concludes.

II. Background

A. Coherence at Cache-Line Granularity

Private caches in a multicore processor are kept coherent by a hardware protocol. The unit tracked by the protocol is the cache line (64 bytes on current x86-64 processors [5]), not the individual variable. Consequently, two variables that reside in the same line are treated by the hardware as a single shared object [6], [7].

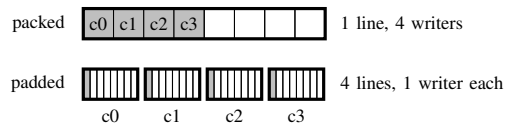


Fig. 1. The two main layouts. Each thick box is one 64-byte cache line divided into eight 8-byte words (padded lines drawn at reduced scale); shaded words hold the counters c0–c3, written by cores 0–3. Packed: 32 bytes of data, one contended line. Padded: 256 bytes, no shared line.

B. The MESI Protocol

In the MESI protocol each line in a private cache is in one of four states [6], [7]:

- **Modified (M):** the only valid copy, which differs from memory;
- **Exclusive (E):** the only cached copy, equal to memory;
- **Shared (S):** one of possibly several read-only copies;
- **Invalid (I):** no usable copy.

A core may write only to a line in state M or E (E changes silently to M). To read a line it does not hold, it issues a read request (BusRd), and any copy in M elsewhere must supply the data and downgrade to S. To write, it issues a read-for-ownership (BusRdX) or, if it already holds the line in S, an upgrade (BusUpgr); in both cases every other copy is invalidated. A write by one core therefore always removes the line from all other caches, and the next access by any of them misses.

Commercial processors implement variants of this protocol. Intel's QuickPath Interconnect protocol, for example, is derived from MESIF, which adds a Forward state to ensure that a single cache answers a request for shared data [8]. How coherence is implemented inside a single client chip such as the i7-1260P is not publicly documented, which is one reason why we use a simple, explicit model alongside the measurements.

C. True Sharing vs. False Sharing

A coherence miss is caused by *true sharing* when the missing core accesses the same data that another core wrote, so the communication is required by the program. It is caused by *false sharing* when the core accesses different data that happen to lie in the same line as the written data; the communication is then an artifact of the line size. Torrellas et al. classified coherence misses in this way and showed that the fraction of false-sharing misses grows with the line size [1].

False sharing hurts when two conditions hold [2]: several cores access the same line, and at least one of the accesses is a write. Fig. 1 shows the case studied in this paper. In the packed layout, four 8-byte counters, each written by a different core, occupy one 64-byte line; every increment invalidates the copies of the other three cores. In the padded layout each counter has its own line and no core ever invalidates another.

D. Detecting False Sharing with `perf c2c`

A HITM (*hit in modified*) is a load that finds the requested line in state M in another core's cache, so the data must be transferred from that cache instead of from the shared cache or memory [9]. Frequent HITM events on one line are the signature of a line that moves between cores.

The `perf c2c` tool, co-developed by Joe Mario and upstream in Linux since version 4.2, records memory accesses through the processor's sampling hardware (PEBS on Intel) and groups them by cache line [3], [9]. For each contended line it reports the number of HITM samples, the offsets within the line that were accessed, the source lines and the CPUs involved. Different threads writing different offsets of one line is the pattern of false sharing; many threads at the same offset is true sharing. Two properties of the tool are relevant for this paper: it is based on sampling, so it sees a fraction of the accesses; and HITM is a property of *load* samples. The report classifies loads by where they were served (L1, L2, LLC, local or remote HITM, DRAM), but classifies stores only as L1 hits, L1 misses, or unknown [9].

III. Related Work and Critical Review of Sources

A. What the Sources Claim

Torrellas, Lam and Hennessy (1994) [1] explain the effect of the cache block size on the miss rate of shared data as the sum of two opposing components: false sharing, which grows with the block size, and spatial locality, which shrinks the miss rate as the block grows. Their analysis of six parallel applications "confirms that false sharing has a significant impact on the miss rate," but also that "poor spatial locality among accesses to shared data has an even larger impact." Their layout optimizations reduced misses on shared data by about 10% on average.

Drepper (2007) [10] measured a program in which each thread increments its own memory location 500 million times. On a machine with four Pentium 4 processors, placing the locations in one line instead of separate lines produced overheads of 390%, 734% and 1147% for two, three and four threads. On a single quad-core Core 2 QX6700, however, the same test "does not show any scalability issues." He also rejects padding everything as a general solution, because the footprint "would increase a lot," and proposes classifying variables by how often they are written.

Liu and Berger (2011) [4] present SHERIFF, which runs threads as processes and compares their per-page updates at synchronization points. They state that false sharing "can degrade application performance by as much as an order of magnitude," report an average detection overhead of 20% (with outliers of $8\times$ and $11\times$), and a speedup of up to $9\times$ from automatic mitigation, on Phoenix and PARSEC benchmarks.

The Linux kernel documentation (2023) [2] defines the two conditions for harmful false sharing, describes common pitfalls (a lock next to the data it protects, small global variables grouped together, members of large structures), and lists mitigations: dedicated cache lines, reorganized structures, reading before writing, and per-CPU counters. It explicitly names a cost of the first mitigation: "more consumption of memory, cache line and TLB entries." It recommends `perf c2c` and `pahole` for detection.

Mario (2016) [3] explains how to use `perf c2c`: a HITM, "a load that hit in a modified cacheline," is "the key that false

sharing has occurred," and the report shows the offsets inside each contended line, which reveal different threads writing different words.

Intel's optimization manual [5] devotes a section to false sharing (Sec. 3.11.5), motivated by a two-socket Xeon system with 224 hardware threads. It recommends `perf c2c`, stating that it "directly measures the cache coherence overhead caused by false sharing," and it lists Mario's blog and the kernel documentation as further reading. On padding, the manual gives two answers. A summary of key practices says: "Place each synchronization variable alone, separated by 128 bytes or in a separate cache line" (p. 11-9), and the L2 streamer prefetcher fetches the pair line of an aligned 128-byte block on "the first access to one of the two cache lines in this block while it is in memory" (p. 3-61). The detailed procedure, however, sets the "false-sharing threshold" to the line size reported by CPUID, with a default of "a safe value 64 bytes" (p. 11-16).

B. Who Wrote Them, and Under What Conditions

None of these sources is suspect a priori, but each was written from a particular position, which limits what its results can show.

- *Hardware generation.* Torrellas et al. simulated 16- and 32-processor bus machines with an invalidation protocol; Drepper measured a Pentium 4 SMP and a Core 2. None of them includes hybrid cores, ring interconnects or modern store buffers. Notably, Drepper's two machines already disagreed with each other, which anticipates our finding that the effect depends on the microarchitecture.
- *Scale.* The kernel documentation motivates the problem with "recent large system with hundreds of CPUs" and gives no slowdown numbers. Mario's guidance also targets large servers ("4, 8 or 16 socket systems") and reports no before/after measurements.
- *Authors and their tools.* The kernel page carries no byline, but the commit that added it shows it was written by an Intel engineer (Feng Tang). Mario co-developed `perf c2c`, and Liu and Berger evaluate their own tool on benchmarks they selected (a dual quad-core Xeon, 32-bit builds). This is normal practice, but it means that the limits of each tool are described less thoroughly than its strengths: SHERIFF, for instance, "cannot currently intercept atomic operations written in assembly" and cannot detect a single writer with readers, and the `perf c2c` documentation does not mention hybrid CPUs.
- *Vendor documents.* The Intel manual describes mechanisms but does not quantify when they cause false sharing, and it recommends both 128 and 64 bytes in different chapters without saying which applies to which processor. Its prefetcher description is written for the Intel Core microarchitecture of 2006, not for current cores.
- *Independence.* The sources cite each other: the Intel manual refers to Mario's blog and to the kernel documentation, which was itself written by an Intel engineer. Their agreement is therefore weaker evidence than it appears.

TABLE I
CRITICAL READING OF THE MAIN SOURCES

Source	Context / possible bias	How we test it
Kernel docs [2]	Intel author; large servers; no numbers	Magnitude on a laptop, 2–4 threads
Mario [3]	Tool co-author; multi-socket servers	HITM detection on P- and E-cores
Intel manual [5]	Vendor; server examples; 128 B and 64 B advice co-exist	64 B vs. 128 B padding; HITM on E-cores
Drepper [10]	2007 hardware; plain increments	Locked vs. plain increments today
Torrellas [1]	Simulated 1990s bus machines	Line-size sweep in our simulator
SHERIFF [4]	Own tool, own benchmarks	Not tested; discussed only

- *Magnitude claims.* "An order of magnitude" (SHERIFF) and the Pentium 4 overheads (Drepper) come from worst-case loops, while Torrellas et al., measuring whole applications, found that false sharing was not the dominant source of misses. Microbenchmarks, including ours, show what false sharing *can* cost, not what it typically costs.

Table I summarizes how we test each source.

IV. METHODOLOGY

A. Workload

Each of N threads increments its own 64-bit counter I times. No counter is ever read or written by another thread, so any coherence traffic between threads is, by construction, false sharing. Four data layouts are compared:

- **packed:** counters stored contiguously (8 B apart), so up to eight counters share one 64-byte line;
- **padded:** counters separated by a stride s (64 or 128 B), so each occupies its own line;
- **local:** each thread accumulates in a private variable and writes the shared array once at the end;
- **shared:** all threads increment the *same* counter (true sharing), used as a reference.

B. Trace-Driven MESI Simulator

We implemented a MESI coherence simulator in C (available in the project repository, `sim/mesi_sim.c`). Its model is deliberately simple so that every result can be traced to a protocol event:

- N cores, each with a private set-associative LRU cache;
- a single, serialized snooping bus carrying BusRd, BusRdX and BusUpgr transactions;
- an atomic increment modeled as a read-for-ownership (BusRdX) followed by a write, and, alternatively, as a separate load and store;
- a per-core clock; at each step the core with the lowest clock issues its next access, so contention emerges from timing instead of from a fixed interleaving;
- an optional *adjacent-line prefetch* that, on a miss, also fetches the other line of the aligned 128-byte pair.

The simulator counts hits, misses, bus transactions, invalidations, cache-to-cache transfers from a modified line (the

TABLE II
SIMULATOR LATENCY PARAMETERS (CYCLES AT 4.7 GHZ)

Event	Cycles	Origin
Private cache hit	4	assumed
Locked RMW, line already owned	18	calibrated
Data from a modified line (HITM)	47	calibrated
Data from LLC / clean copy	40	assumed
Main memory	200	assumed
Upgrade (S→M)	30	assumed

TABLE III
SIMULATED NS PER INCREMENT, 4 THREADS (PACKED COUNTERS AT DISTANCE d)

	Distance between counters d (bytes)					
Line	8	16	32	64	128	256
32 B	40.0	40.0	4.1	4.1	4.1	4.1
64 B	40.0	40.0	40.0	4.1	4.1	4.1
128 B	40.0	40.0	40.0	40.0	4.1	4.1
256 B	40.0	40.0	40.0	40.0	40.0	4.1

analogue of a HITM), write-backs, and the memory occupied by the counters. Latencies are model parameters (Table II); a sensitivity sweep over the HITM latency is also reported.

The two parameters that dominate the workload were calibrated from our own measurements on the P-cores (Section V): the cost of an uncontended locked increment (padded layout, one thread, 3.87 ns) and the cost of one line transfer, taken from the slope of the packed layout with four threads (about 10 ns per transfer). The remaining values are rough estimates; they barely affect this workload because, after warm-up, it never misses to the LLC or to memory. Note that calibrating with data from the same platform means that agreement at those two points is not an independent validation; only the predictions for other thread counts and layouts are.

Although the i7-1260P connects its cores through a ring and a shared L3 rather than a bus, the serialized bus is adequate for this workload: only one line is contended, and only one core can hold it in state M at a time, so its transfers are serialized whatever the interconnect. The model would be inadequate for workloads with many independent contended lines.

Four sweeps are run (`sim/sweep.sh`): (i) scaling with N for each layout; (ii) line size versus counter stride; (iii) adjacent-line prefetch with 64-byte and 128-byte padding; and (iv) sensitivity to the HITM latency.

C. Hardware Experiment

The experiment runs on an Intel Core i7-1260P (Alder Lake) with four performance cores (P-cores, two hardware threads each, private 1.25 MiB L2, up to 4.7 GHz), eight efficiency cores (E-cores, up to 3.4 GHz, organized in two clusters of four that share a 2 MiB L2) and an 18 MiB L3 shared by all cores, under Linux 7.0 with the `performance` frequency governor. The benchmark (`benchmarks/fs_bench.c`, compiled with GCC 13.3 at `-O2`) implements the four layouts of the simulator, with either a locked increment (`__atomic_fetch_add`, which compiles to `lock add`) or a plain `volatile` increment. Each thread is pinned to one core, and three groups of cores are compared: one thread per physical P-core (CPUs 0, 2, 4, 6), four E-cores of one cluster (8–11), and E-cores split across both clusters (8, 12, 9, 13). Each configuration performs 5×10^7 increments per thread and is repeated ten times after one warm-up run. Contended lines are recorded with `perf c2c record` for each layout and core group.

V. RESULTS

A. Simulation

Table III shows the simulated cost of an increment with four threads as a function of the cache-line size and the distance between counters. The result is binary: whenever two counters fall in the same line, every increment pays a line transfer; as soon as the distance reaches the line size, the cost drops to that of an uncontended locked increment. The boundary moves with the line size, which reproduces in our model the argument of Torrellas et al. [1] that longer lines increase false sharing.

With the adjacent-line prefetch enabled and 64-byte padding, the simulator records only one to four prefetch-induced HITM events per run (for 2 to 8 threads), all during warm-up. Once each core owns its line in the M state it never misses again, so the prefetcher is never triggered. The sensitivity sweep confirms that, in the model, the cost of the packed layout is set entirely by the assumed transfer latency: with four threads it equals four times that latency (120 to 600 cycles for latencies of 30 to 150 cycles), while padded stays at 19 cycles. This is why the transfer latency had to be calibrated rather than assumed.

B. Measurements on the i7-1260P

Fig. 2 shows the cost per increment for each layout and core group with locked increments. With one thread all layouts except local cost the same (3.87 ns on P-cores, 5.32 ns on E-cores; about 18 cycles in both cases). With two or more threads, the packed and shared layouts grow roughly linearly with the thread count, while padded remains almost flat. With four threads, packed is $8.4\times$ slower than padded on P-cores, $9.2\times$ on E-cores sharing an L2, and $11.6\times$ on E-cores in different clusters (Fig. 3). Padding to 128 bytes was within 5% of 64-byte padding in every configuration and is omitted from the figure. All configurations were repeated ten times; the coefficient of variation was below 2% except for the local layout, whose runs last only a few milliseconds.

The type of increment changes the picture. With plain `volatile` increments (a load, an add and a store, without `lock`), the packed layout is only $1.15\times$ slower than padded on P-cores, but $3.8\times$ to $5.5\times$ slower on E-cores (Fig. 3). On a P-core a single thread completes about one `volatile` increment per cycle.

Table IV summarizes the `perf c2c` recordings. On P-cores, the tool reports one contended line with 18 631 HITM samples and shows each thread writing at a different offset (0x0, 0x8, 0x10, 0x18), exactly the pattern described by

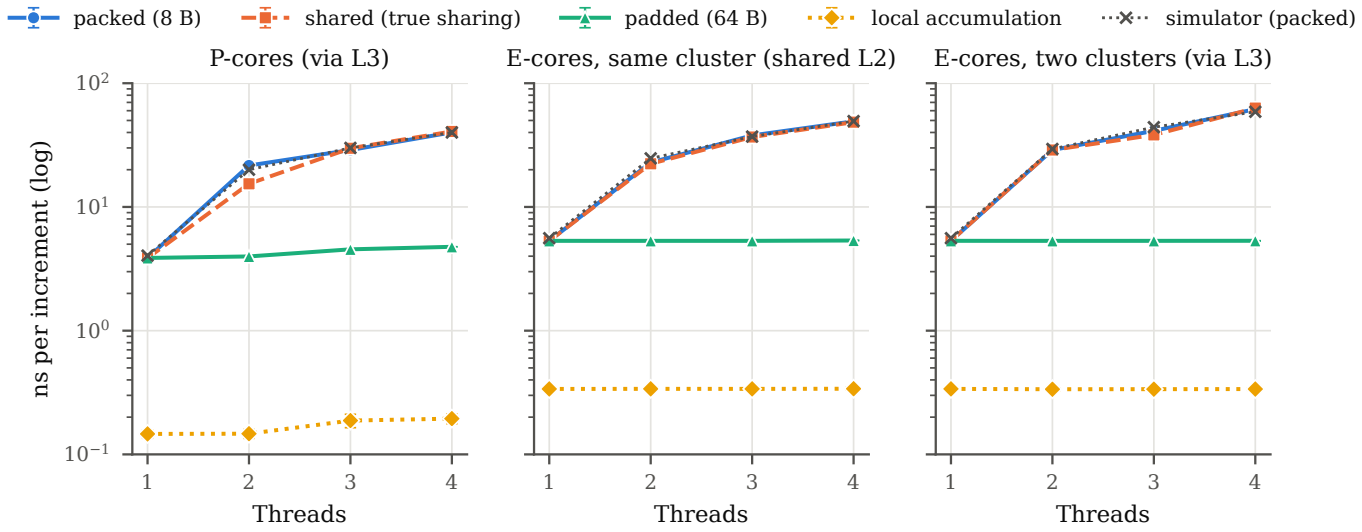


Fig. 2. Cost per locked increment versus thread count on the i7-1260P (mean of 10 runs; error bars show one standard deviation, mostly smaller than the markers). Each panel pins the threads to a different group of cores. The dotted line is the prediction of our MESI simulator for the packed layout, using the line-transfer latency calibrated for each group (Table II).

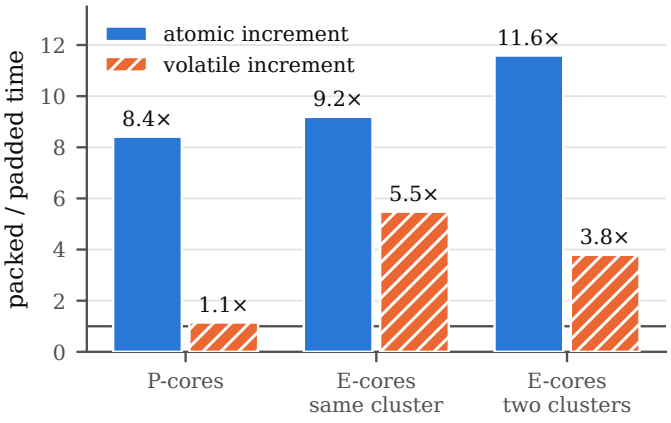


Fig. 3. Slowdown of the packed layout relative to 64-byte padding with four threads, for locked (`lock add`) and plain `volatile` increments.

TABLE IV
HITM SAMPLES REPORTED BY `perf c2c`, 4 THREADS

Layout	P-cores	E same	E cross
packed, atomic	18 631	54	293
shared, atomic	12 951	186	97
padded, atomic	0	0	1
local	0	0	0
packed, volatile	0	0	20
Measured slowdown, packed vs. padded (atomic)			
	8.4×	9.2×	11.6×

Mario [3]. Padded and local layouts produce no HITM. However, the same packed workload on E-cores yields only 54 and 293 HITM samples, and the `volatile` packed layout yields almost none on any group.

C. Simulation vs. Hardware

After calibrating two parameters per core group, the simulator predicts the packed layout within ±7% for two to four threads in all three groups (Fig. 2). The fitted line-transfer cost is 47 cycles on P-cores, 42 cycles between E-cores sharing an L2, and 50 cycles between E-core clusters. The model fails in two places: it predicts that packed and shared cost the same, whereas on two P-cores packed is 40% slower than shared (21.6 vs. 15.4 ns); and, since it has no store buffer, it cannot reproduce the near absence of false sharing for `volatile` increments on P-cores. Both discrepancies are discussed in Section VI.

VI. DISCUSSION

Each subsection below starts from a claim found in the sources, contrasts it with what we simulated and measured, and states which parts of our explanation are confirmed and which remain hypotheses.

A. False Sharing Costs as Much as True Sharing

False sharing is usually presented as a subtler, secondary form of sharing. Our data do not support that distinction from the point of view of the hardware. In the simulator, the packed and shared layouts generate exactly the same sequence of coherence transactions, because the protocol tracks lines and not words. On the real processor, both layouts cost within 8% of each other (within 3% in five of the six cases) for three and four threads in all core groups (Fig. 2). The practical consequence is that a programmer who removes an obviously shared variable but leaves independent counters in one line gains nothing.

The only exception is two threads on P-cores, where packed is 40% slower than shared (21.6 vs. 15.4 ns). Our model cannot reproduce this, and we have no confirmed explanation. Possible causes include a different arbitration of ownership requests when the two threads target different words of the line, or a difference in how often each core keeps the line for several consecutive increments. Verifying either would require

counters that we have not collected; we report the discrepancy as an open observation.

B. Does 128-Byte Padding Matter?

Intel's manual recommends separating synchronization variables by 128 bytes, and its L2 streamer fetches the pair line of an aligned 128-byte block [5]; practitioners often recommend 128-byte padding for that reason. The same manual, however, defaults to 64 bytes elsewhere. In our measurements, 128-byte padding was within 5% of 64-byte padding in every configuration, and slightly slower in some. The simulator suggests why, and the manual's own wording agrees: the streamer is triggered by an access to a line "while it is in memory," that is, on a miss. Once each core owns its padded line in state M it never misses again, so the neighboring line is fetched at most once during warm-up (one to four events per run). The recommendation is therefore not wrong but conditional: it should matter when the contended lines are repeatedly evicted or re-fetched, for example when the counters are mixed with other data that miss in the cache. For a tight update loop such as ours, it doubles the memory cost without a measurable benefit. The manual states the mechanism but does not quantify when the extra padding pays off, and its two recommendations are not reconciled; our results show that it is a workload-dependent decision.

C. The Classic `volatile` Demonstration Can Hide the Problem

Many explanations of false sharing, including our own first prototype, use a loop of `volatile` increments. On P-cores, this demonstration nearly fails: packed is only $1.15\times$ slower than padded, while with locked increments the same layout is $8.4\times$ slower. The difference is that a `lock add` must own the line for every single increment, whereas a plain store can wait in the store buffer. Our hypothesis is that on the P-core the thread reads its own pending stores (store-to-load forwarding), so its loads rarely need the line, and the buffered stores drain in bursts whenever the core obtains ownership, amortizing one transfer over many increments. The fact that a single P-core thread completes about one dependent `volatile` increment per cycle suggests an aggressive forwarding or memory-renaming mechanism. On E-cores the same code is $3.8\times$ to $5.5\times$ slower when packed, so the effect is microarchitecture-specific.

Fast forwarding alone, however, does not explain the difference. Ertl's memory-dependency microbenchmark [11] measures dependent store-load chains on both core types of a hybrid Core i3-1315U and finds about 0.65 cycles per operation on the P-core (Golden Cove) and 0.7 to 1 cycle on the E-core (Gracemont): both forward a store to a dependent load with almost no latency. The difference must therefore lie in how each core drains its store buffer while another core holds the line, which we cannot observe with our tools. We keep this part of the explanation as a hypothesis.

The observation is not new. Drepper's 2007 experiment used plain (non-atomic) increments: it showed overheads of up to 1147% on a Pentium 4 multiprocessor but no scalability problem on a quad-core Core 2 [10]. Nineteen years later, the same kind of program still gives opposite answers depending on the core it runs on, now within a single chip.

This matters for the sources: a demonstration that works on one core type and not on another, of the same chip, cannot serve as evidence of the magnitude of the problem. It also means that the cost of false sharing depends as much on the instruction that writes the data as on the layout.

D. Limits of HITM-Based Detection

Mario [3] presents `perf c2c` as a way to find false sharing through HITM events, and our P-core recordings fully confirm that description: one line, four offsets, one source line. However, our results expose two blind spots that the tool's documentation does not emphasize.

First, on E-cores the tool reports between 60 and 350 times fewer HITM samples for a slowdown of the same order. In the recordings, E-cores produce almost no load samples (165 versus 44 634 on P-cores) and the locked increments appear mainly as store samples, which `perf c2c` classifies only as L1 hits or misses [9], so they can never count as HITM. Because `perf c2c` ranks lines by HITM count, a line that is equally harmful but accessed from E-cores could be ranked below less harmful lines in a real program. Second, the `volatile` packed layout produces almost no HITM on any core group, although on E-cores it is up to $5.5\times$ slower than padded. This is the most dangerous combination: a real cost that the tool does not show.

This qualifies Intel's statement that the tool "directly measures" the coherence overhead [5]: it measures a sample of loads, and on E-cores that sample is small. We do not claim that the tool is wrong; it reports what the sampling hardware provides. But its effectiveness depends on the core type and on the access pattern, and that dependence is not visible to the user. The fact that the author of the documentation is also a co-author of the tool is not, by itself, evidence of bias; what our data show is that its description was written for server processors without hybrid cores, which did not exist at the time.

E. Generalizing from Servers to a Hybrid Laptop

The kernel documentation [2], written by an Intel engineer, draws its examples from regressions found on large servers. One could suspect that false sharing is a problem only at that scale. Our results show otherwise: two threads on a laptop already suffer a $5.4\times$ slowdown. At the same time, topology matters even inside one chip. A line transfer between E-cores that share an L2 costs about 42 cycles, compared with 50 cycles between E-core clusters and 47 cycles between P-cores through the L3.

The slowdown factors themselves deserve caution. The E-cores show the largest factor ($11.6\times$), but partly because their uncontended locked increment is slower in nanoseconds; in cycles, the uncontended cost is the same (18 cycles) on both core types. Reporting only "times slower," as is common, mixes the cost of the transfer with the cost of the baseline. We therefore also report the absolute cost per transfer, which is the quantity that the simulator predicts.

F. The Cost of Mitigation

Padding a counter from 8 to 64 bytes multiplies its memory footprint by eight, and to 128 bytes by sixteen. For four counters this is irrelevant (32 B versus 256 B), but for per-object counters in large arrays or structures it wastes cache capacity and TLB reach, which can slow down the rest of the program. The best layout in our experiment was not padding but per-thread accumulation, which was 16 (E-cores) to 27 (P-cores) times faster than padding and did not need extra memory. It corresponds to the per-CPU counters recommended by the kernel documentation [2]. Padding is a fallback for data that must remain shared; restructuring the computation to avoid writing shared lines is the stronger mitigation.

G. Threats to Validity

- **Worst-case workload.** The microbenchmark does nothing but update counters. Real programs interleave computation, so the slowdowns we report are upper bounds for this access pattern.
- **Calibration.** The simulator was calibrated with data from the same machine; its agreement at the calibration points is not an independent validation.
- **Simplified model.** The simulator uses a serialized bus, has no store buffer and no ring topology, and models the prefetcher only approximately.
- **Frequency.** Turbo frequency decreases with the number of active cores; this probably explains why padded rises from 3.87 to 4.76 ns on P-cores, and it affects conversions between nanoseconds and cycles.
- **One platform, one sample of `perf c2c`.** All results come from one processor, and each `perf c2c` configuration was recorded once.

VII. Conclusions

RQ1 (magnitude). On the i7-1260P, placing four independent counters in one cache line made locked increments $8.4\times$ to $11.6\times$ slower than padding them, depending on the cores involved, and two threads were already enough for a $5.4\times$ slowdown on P-cores. The cost grows linearly with the number of threads, about one line transfer per increment, which costs 42 to 50 cycles depending on whether the cores share an L2. Hardware does not distinguish false from true sharing: both cost the same.

RQ2 (claims of the sources). The basic mechanism described by all the sources was confirmed by both the simulator and the measurements. Three commonly repeated recommendations, however, depend on context. Padding to 128 bytes brought no benefit over 64 bytes for our workload, because the adjacent-line prefetcher acts only on misses. The classic `volatile` demonstration nearly hides the problem on P-cores but not on E-cores of the same chip. And `perf c2c`, which works exactly as documented on P-cores, reports 60 to 350 times fewer HITM events on E-cores and almost none for the `volatile` pattern, even when the slowdown is real. None of these sources is wrong; each was written for a context (server processors, homogeneous cores, locked updates) that does not cover a hybrid laptop.

RQ3 (cost of mitigation). Padding multiplies the memory of a counter by 8 (64 B) or 16 (128 B). It is negligible for a few hot counters but not for per-object fields in large data structures. Per-thread accumulation was 16 to 27 times faster than padding and needed no extra memory.

From these results we derive four practical guidelines: (1) prefer per-thread or per-CPU accumulation over padding; (2) pad hot, frequently written data to 64 bytes, and consider 128 bytes only if measurements show a benefit; (3) do not use `volatile` microbenchmarks to estimate the cost of false sharing; (4) on hybrid processors, do not interpret a low HITM count in `perf c2c` as evidence that there is no false sharing, especially for threads running on E-cores.

As future work, the simulator could add a store buffer to model the `volatile` case, the experiment could be repeated on an AMD processor, whose `perf c2c` support relies on a different sampling mechanism, and the analysis could be extended from a microbenchmark to a real application.

References

[1] J. Torrellas, M. S. Lam, and J. L. Hennessy, "False sharing and spatial locality in multiprocessor caches," *IEEE Transactions on Computers*, vol. 43, no. 6, pp. 651–663, Jun. 1994.

[2] F. Tang, "False sharing," The Linux Kernel Documentation, Kernel Hacking Guides, 2023, accessed: Nov. 5, 2026. [Online]. Available: https://kernel.org/doc/html/latest/kernel-hacking/false-sharing.html

[3] J. Mario, "C2C – false sharing detection in Linux perf," Sep. 2016, accessed: Nov. 5, 2026. [Online]. Available: https://joemario.github.io/blog/2016/09/01/c2c-blog/

[4] T. Liu and E. D. Berger, "SHERIFF: Precise detection and automatic mitigation of false sharing," in *Proc. ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, Portland, OR, USA, 2011, pp. 3–18.

[5] Intel Corporation, *Intel 64 and IA-32 Architectures Optimization Reference Manual, Volume 1*, Dec. 2023, rev. 049. Accessed: Nov. 12, 2026. [Online]. Available: https://www.intel.com/content/www/us/en/content-details/814198/intel-64-and-ia-32-architectures-optimization-reference-manual-volume-1.html

[6] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.

[7] V. Nagarajan, D. J. Sorin, M. D. Hill, and D. A. Wood, *A Primer on Memory Consistency and Cache Coherence*, 2nd ed., ser. Synthesis Lectures on Computer Architecture. Springer, 2020.

[8] J. R. Goodman and H. H. J. Hum, "MESIF: A two-hop cache coherency protocol for point-to-point interconnects," University of Auckland, Tech. Rep., 2009, manuscript submitted to ISCA 2010. Accessed: Nov. 12, 2026. [Online]. Available: https://www.cs.auckland.ac.nz/~goodman/TechnicalReports/MESIF-2009.pdf

[9] Linux perf developers, "perf-c2c(1) – shared data C2C/HITM analyzer," Linux manual page, accessed: Nov. 5, 2026. [Online]. Available: https://man7.org/linux/man-pages/man1/perf-c2c.1.html

[10] U. Drepper, "What every programmer should know about memory," Red Hat, Inc., Tech. Rep., Nov. 2007. [Online]. Available: https://people.freebsd.org/~lstewart/articles/cpumemory.pdf

[11] M. A. Ertl, "Memory dependency microbenchmark," TU Wien, accessed: Nov. 12, 2026. [Online]. Available: https://www.complang.tuwien.ac.at/anton/memdep/